Project Report: Assignment 1

Damaris Stephanie Ndjebayi, McGill University, School of Computer Science
Noé Vartanian, McGill University, School of Computer Science

February 5, 2026

Abstract

In this project, we predict the total number of daily rented bikes (*cnt*) using the UCI Bike Sharing *day.csv* dataset. We perform exploratory data analysis to guide preprocessing and model design, and implement linear regression from scratch using a numerically stable least-squares solver. To improve model capacity, we investigate non-linear feature engineering through polynomial and interaction terms. The baseline linear regression model achieves a test MSE of 668,421.25 and an R^2 score of 0.802, while the best feature-engineered model reduces the test MSE to 506,988.56 and increases the test R^2 to 0.850, demonstrating the benefit of incorporating simple non-linear transformations.

1 Introduction

Predicting daily bike rental demand is a classical regression problem that combines temporal, weather-related, and calendar-based factors. In this project, we study the Bike Sharing Dataset from the UCI Machine Learning Repository and focus on modeling the total number of rented bikes per day. Rather than emphasizing the dataset structure in detail, we use this task as a case study to examine how data preprocessing, modeling assumptions, and feature engineering affect regression performance. We address this problem using linear regression, implemented from scratch in Python with an analytical closed-form solution and a numerically stable least-squares solver. This approach allows us to clearly analyze the behavior and limitations of linear models without relying on external machine learning libraries. The main objectives of this project are threefold: to explore and preprocess real-world data using exploratory analysis, to implement linear regression using its analytical formulation, and to investigate how simple feature engineering techniques influence model fit and generalization.

2 Data Preprocessing and Exploration

We begin by assessing data quality and consistency. All 731 observations are complete, with no missing values across the 16 original features, and all variables fall within their expected domains. Since no anomalies are detected, no imputation or row removal is required.

To avoid data leakage and redundancy, we remove features that are not suitable for prediction. The variable *instant* is dropped as it is only an index, while *casual* and *registered* are excluded because their sum exactly equals the target variable *cnt*. The date variable *dteday* is also removed, as its information is already captured by calendar-related features such as *season*, *mnth*, and *yr*.

Exploratory data analysis is conducted using visualizations to better understand relationships between features and the target. Box plots of categorical variables (Figure 1) show that bike rental demand varies significantly across categories, with lower usage under poor weather conditions and higher usage during warmer seasons, justifying the inclusion of these variables in the model.

Analysis of numerical features using a pairwise scatter matrix (Figure 2) reveals a near-perfect linear relationship between *temp* and *atemp*, indicating strong multicollinearity. Since linear regression is sensitive to correlated inputs, *atemp* is removed and *temp* is retained to reduce redundancy.

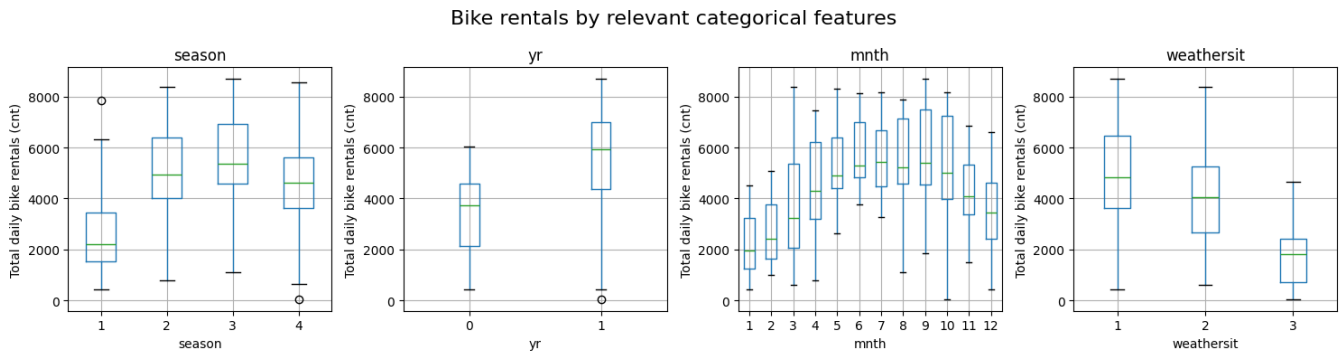

Figure 1: Box plots of categorical features versus the target variable

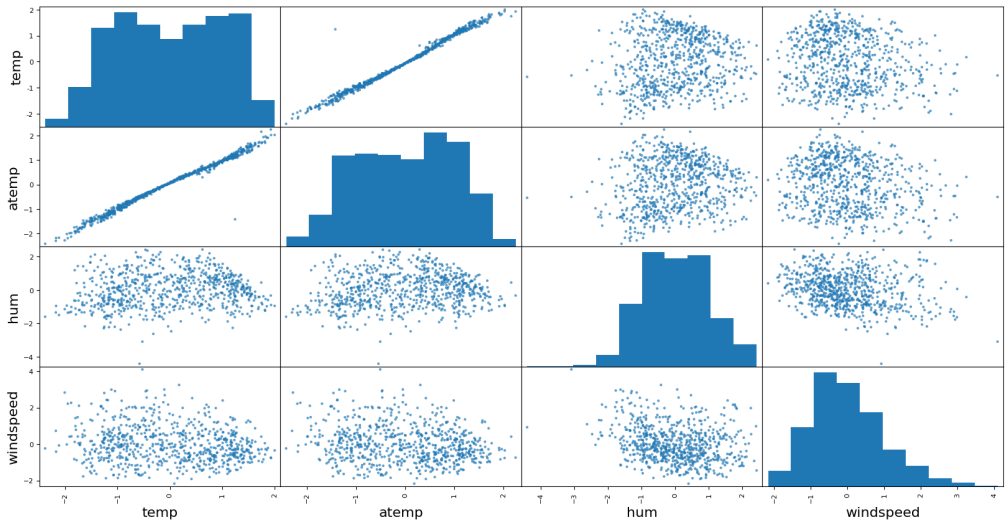

Figure 2: Pairwise scatter matrix of continuous features

Categorical variables with more than two levels are encoded using one-hot encoding because they are nominal and do not possess an inherent ordering, while binary variables are left unchanged. Finally, continuous features are standardized using statistics computed on the training set only, improving numerical stability and preventing data leakage.

3 Methods

3.1 Linear Regression

We model daily bike rental demand using linear regression. Given a feature matrix $X \in \mathbb{R}^{N \times d}$ and a target vector $y \in \mathbb{R}^N$, the model assumes a linear relationship

$$\hat{y} = Xw + b,$$

where w denotes the model parameters and b an intercept term. The intercept is incorporated by appending a column of ones to the feature matrix.

Model parameters are estimated by minimizing the mean squared error objective using an ordinary least squares formulation. To ensure numerical stability and avoid explicit matrix inversion, we compute the solution using the NumPy least-squares solver (`np.linalg.lstsq`), which is robust to correlated features.

Since the original dataset is ordered chronologically, the data is randomly shuffled prior to splitting to prevent bias induced by temporal ordering. We then apply an 80/20 train–test split, resulting in 584

training samples and 147 test samples. With 32 input features, this split provides sufficient data to reliably estimate model parameters while preserving a representative test set.

Model performance is evaluated primarily using Mean Squared Error (MSE), which directly corresponds to the optimization objective of linear regression. Mean Absolute Error (MAE) and the coefficient of determination (R^2) are also reported to provide complementary and interpretable performance measures.

3.2 Feature Engineering

To increase the expressive capacity of the linear regression model while maintaining interpretability, we apply several non-linear feature engineering techniques. Performance before and after feature engineering is compared using MSE, MAE, and R^2.

First, polynomial features of degree two are added for numerical variables to allow the model to capture simple non-linear relationships between individual features and the target, while limiting the risk of overfitting associated with higher-degree expansions. Second, interaction terms are introduced to model pairwise dependencies between selected continuous variables, specifically *temp* × *hum*, *temp* × *windspeed*, and *hum* × *windspeed*.

Finally, we evaluate a monotonic non-linear transformation using the inverse hyperbolic sine (*asinh*) applied to *hum* and *windspeed*. This transformation behaves similarly to a logarithmic mapping while remaining well-defined for negative values and serves as an ablation study to assess the effect of non-linear preprocessing beyond polynomial and interaction terms.

4 Results

4.1 Linear Regression before the Feature engineering

We presented in the table 1 the results of Linear Regression before applying any feature engineering techniques.

Table 1: Performance of the linear regression model on training and test datasets before the Feature engineering

Metric	**Training Set**	**Test Set**
Mean Squared Error (MSE)	552,949.85	668,421.25
Mean Absolute Error (MAE)	544.92	587.07
R^2 Score	0.856	0.802

4.2 Linear Regression after the Feature engineering

4.2.1 Comparison for the training MSE in both setting

The table 2 shows the results obtained with the different metrics on the training set.

Table 2: Performance of linear regression models on the training dataset

Method	**MSE**	**MAE**	R^2
Baseline (no feature engineering)	552,949.85	544.92	0.856
Asin transformation	552,949.85	544.92	0.856
Interaction features	548,263.52	540.46	0.857
Polynomial features	472,888.53	505.67	0.877
Polynomial + Interaction	**462,993.92**	**500.97**	**0.879**

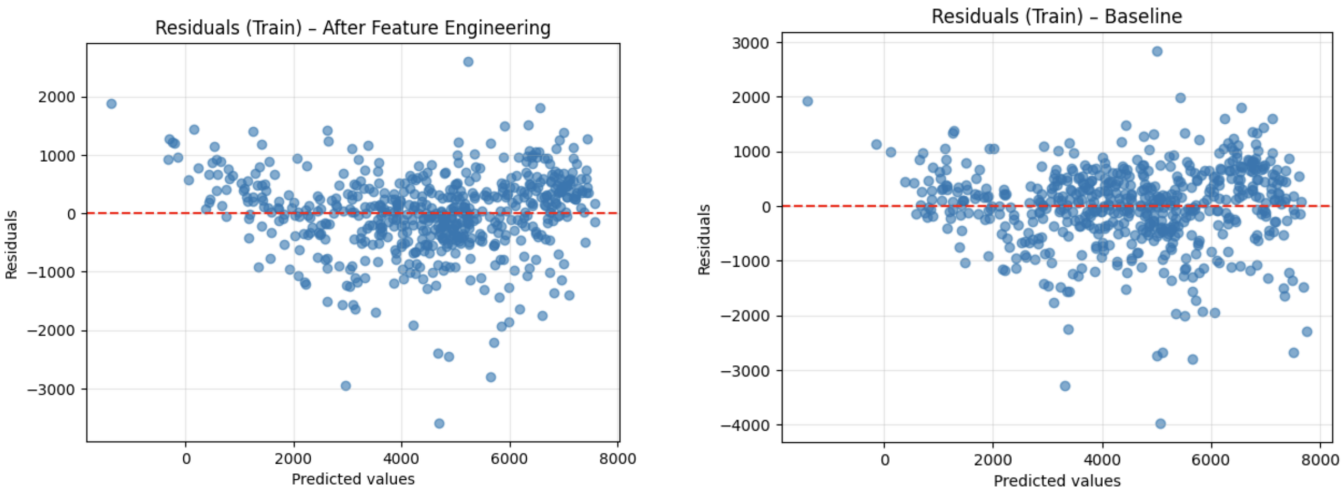

Figure 3: Residuals versus predicted values on the training set before and after feature engineering.

4.2.2 Comparison for the Test MSE in both setting

The table 3 shows the results obtained with the different metrics on the test set

Table 3: Performance of linear regression models on the test dataset

Method	**MSE**	**MAE**	R^2
Baseline (no feature engineering)	668,421.25	587.07	0.802
Asin transformation	668,421.25	587.07	0.802
Interaction features	669,275.88	587.70	0.802
Polynomial features	510,114.43	545.41	0.849
Polynomial + Interaction	**506,988.56**	**545.69**	**0.850**

5 Discussion and Conclusion

Effect of preprocessing on model performance and stability Preprocessing played a crucial role in ensuring numerical stability and meaningful model evaluation. Removing leaky features (*casual* and *registered*) was essential to preserve the validity of the regression task, as their inclusion would have led to artificially inflated performance. Eliminating the identifier feature *instant* and the highly correlated variable *atemp* reduced redundancy and mitigated multicollinearity, which is known to destabilize parameter estimates in linear regression. Standardizing continuous variables (*temp*, *hum*, *windspeed*) further improved numerical conditioning and prevented features with larger magnitudes from dominating the optimization. Importantly, applying scaling parameters computed only on the training set avoided data leakage and ensured consistent transformations across splits. Overall, these preprocessing choices resulted in more stable parameter estimates and reliable performance evaluation.

Most informative features Feature importance was assessed using the magnitude of the learned regression coefficients. The most influential variables include *yr*, *weathersit*, *season*, and *temp*. These results align with domain knowledge: bike usage increases under favorable weather conditions and warmer temperatures, and exhibits a clear upward trend over time as the bike-sharing system becomes more widely adopted. Figure 4 illustrates the relative importance of the most informative features.

Effects of non-linear feature engineering on training error Polynomial feature transformations led to a substantial reduction in training error. Compared to the baseline model, polynomial features reduced the training MSE from 552,949.85 to 472,888.53 (approximately 14.5%). Combining polynomial

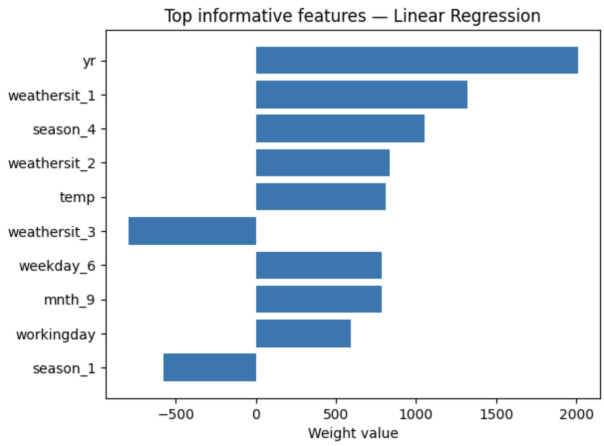

Figure 4: Top 10 most informative features based on regression coefficients

and interaction features further reduced the training MSE to 462,993.92, yielding the lowest training error among all evaluated models. This demonstrates the presence of meaningful non-linear relationships that cannot be captured by a purely linear model.

In contrast, interaction features alone provided only marginal improvements, reducing the training MSE to 548,263.52. Similarly, applying the *asinh* transformation resulted in performance identical to the baseline model (Table 2), indicating that this monotonic non-linear transformation did not introduce additional predictive information.

Limitations of linear regression on this dataset Despite performance improvements, several limitations remain. First, even with feature engineering, linear regression is limited in its ability to capture complex or highly localized non-linear patterns, as it remains linear in its parameters and relies on predefined transformations. Second, expanding the feature space with polynomial and interaction terms increases the risk of feature collinearity, which can lead to unstable parameter estimates and reduced generalization. Finally, residual analysis (Figure 3) reveals the presence of several large residuals, highlighting the model's sensitivity to outliers. Ordinary least squares regression is particularly vulnerable to such points, which can disproportionately influence the fitted coefficients and reduce overall robustness.

In this project, we demonstrated the importance of careful data preprocessing and exploratory analysis when applying linear regression to real-world data. Implementing the model from scratch allowed us to clearly analyze the effects of preprocessing choices and modeling assumptions on performance and stability. While non-linear feature engineering significantly improved model fit, the remaining limitations highlight the need for more expressive models or regularization techniques for further performance gains.

6 Statement of Contributions

Noé: brainstorming, writing (Abstract, Introduction, Data preprocessing and Exploration, Method), coding. **Damaris**: brainstorming, writing (Method, Results, Discussion and Conclusion), coding.

7 Statement on the Use of LLMs and Other Resources

Large Language Models (LLMs) were used as a writing and editing aid to improve clarity, structure, and language quality of the report. The LLM was also used to assist with code organization and debugging suggestions. All modeling decisions, data analysis, experimental design, implementation, and interpretation of results were conducted by the authors. No external solutions or answers were copied directly, and all conclusions reflect the authors' own understanding and work.